# WAMO Labs — Interview Cheatsheet

MERN / Python / AI-ML Role | Faizan Yousaf

## 1. OOP — 4 Pillars + Composition vs Aggregation

| Pillar | One-liner | JS Example |
|---|---|---|
| Encapsulation | Hide data, expose interface | Private fields #name; getter/setter |
| Abstraction | Hide complexity, show essentials | Abstract class / interface pattern |
| Inheritance | Child reuses parent behavior | class Dog extends Animal |
| Polymorphism | Same call, different behavior | speak() differs per subclass |

### Composition vs Aggregation

| | Composition | Aggregation |
|---|---|---|
| **Relationship** | "Has-A" (strong ownership) | "Has-A" (weak ownership) |
| **Lifecycle** | Child dies with parent | Child lives independently |
| **Box diagram** | Class B holds instance of A<br>→ A cannot exist without B | B references A<br>→ A can exist alone |
| **JS Example** | class Car { engine = new Engine() } | class Team { constructor(player){} } |

### Inheritance — Base/Derived calling

```
class Animal { speak() { return 'generic sound'; } }
class Dog extends Animal { speak() { return super.speak() + ' + bark'; } }
```

## 2. JavaScript — var / let / const, Hoisting, Async

| | var | let | const |
|---|---|---|---|
| | Function | Block | Block |
| Hoisting | Yes (undefined) | Yes but TDZ | Yes but TDZ |
| | Yes | No | No |
| Re-assign | Yes | Yes | No |

### Temporal Dead Zone (TDZ)

• let/const are hoisted but NOT initialized until declaration line

• Accessing them before declaration = ReferenceError

```
console.log(x); // ReferenceError — TDZ
let x = 5;
```

### Promise.all vs Promise.race

| | Promise.all | Promise.race |
|---|---|---|
| | ALL promises resolve | FIRST promise settles |
| Rejects when | ANY one rejects | First settled is rejection |
| | Multiple APIs, all needed | Timeout / fastest wins |
| Returns | Array of all results | Single result |

### Scenario: Call APIs sequentially

```
// Sequential (next only if previous resolves)
const r1 = await fetch('/api1');
const r2 = await fetch('/api2'); // only runs after r1

// Parallel (all at once, wait for all)
const [r1, r2] = await Promise.all([fetch('/api1'), fetch('/api2')]);
```

## 3. DBMS — Normalization, Indexing, ACID, Transactions

### Normalization vs Denormalization

| | Normalization | Denormalization |
|---|---|---|
| | Remove redundancy | Improve read speed |
| Best for | Write-heavy systems | Read-heavy systems |
| | More JOINs needed | Data duplication |
| Example | OLTP (banking) | Analytics / dashboards |

### Normal Forms — Quick Reference

| Form | Rule |
|---|---|
| 1NF | Atomic values, no repeating groups |
| 2NF | 1NF + No partial dependency on composite PK |
| 3NF | 2NF + No transitive dependency (non-key → non-key) |
| BCNF | Every determinant is a candidate key |

### ACID Properties — CRITICAL

| Property | Meaning | Example |
|---|---|---|
| Atomicity | All or nothing — transaction fully completes or fully rolls back | Bank transfer: debit + credit both happen or neither does |
| Consistency | DB moves from one valid state to another; all rules/constraints satisfied | Balance can never go negative after transaction |
| Isolation | Concurrent transactions don't interfere with each other | Two users booking same seat — only one succeeds |
| Durability | Committed data persists even after crash/power failure | Order confirmed → saved even if server crashes next second |

### Transactions

• A transaction = a unit of work that must complete fully or not at all

• BEGIN TRANSACTION → execute SQL → COMMIT (save) or ROLLBACK (undo)

```
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT; -- or ROLLBACK if any error
```

### Isolation Levels (low → high protection)

| Level | Dirty Read | Non-Rep. Read | Phantom Read |
|---|---|---|---|
| Read Uncommitted | YES | YES | YES |
| Read Committed | No | YES | YES |
| Repeatable Read | No | No | YES |
| Serializable | No | No | No |

### Multiplicity & Indexing

• 1:1 — one person has one passport

• 1:M — one user has many orders

• M:N — many students enroll in many courses → needs junction table

• Indexing: speeds up SELECT/WHERE queries — costs extra write time

• Use indexes on: columns used in WHERE, JOIN, ORDER BY — avoid on rarely-queried columns

## 4. Scenario: InDrive-type Schema + Query

### Tables

```
Rider(rider_id PK, name, phone, rating)
```

```
Driver(driver_id PK, name, phone, vehicle, rating, is_available)
Booking(booking_id PK, rider_id FK, driver_id FK, pickup, dropoff,
status, price, created_at)
-- status: 'pending','accepted','completed','cancelled'
```

### Relationships

• Rider 1:M Booking — one rider makes many bookings

• Driver 1:M Booking — one driver completes many bookings

• Rider M:N Driver (via Booking junction table)

### Sample Query — All completed rides for a rider

```
SELECT b.booking_id, d.name AS driver, b.pickup, b.dropoff, b.price
FROM Booking b
JOIN Driver d ON b.driver_id = d.driver_id
WHERE b.rider_id = 101 AND b.status = 'completed'
ORDER BY b.created_at DESC;
```

## 5. DSA — Reverse Number, Valid Parenthesis, Linked List Sort

### Reverse a Number

```
function reverseNum(n) {
let rev = 0, num = Math.abs(n);
while (num > 0) { rev = rev * 10 + num % 10; num = Math.floor(num/10); }
return n < 0 ? -rev : rev;
}
```

### Valid Parenthesis (Stack)

```
function isValid(s) {
const stack = [], map = {')':'(',']':'[','}':'{'};
for (let c of s) {
if ('([{'.includes(c)) stack.push(c);
else if (stack.pop() !== map[c]) return false;
}
return stack.length === 0;
}
```

### Linked List of Lists — Sort (O(n²) ok)

• Each node: { val, next, innerList }

• Flatten all values from all inner lists into one array

• Sort the array

• Rebuild linked list from sorted array

```
// Flatten step
function flatten(head) {
const vals = [];
while (head) {
let inner = head.innerList;
while (inner) { vals.push(inner.val); inner = inner.next; }
head = head.next;
}
return vals.sort((a,b) => a-b);
}
```

## 6. Quick Recall — Don't Blank On These

| Topic | Say This |
|---|---|
| **ACID** | Atomicity, Consistency, Isolation, Durability |
| **TDZ** | let/const hoisted but uninitialized — access before decl = ReferenceError |
| **Promise.all** | Waits for ALL — one rejection kills all |

| | |
|---|---|
| **Promise.race** | First to settle wins — resolve OR reject |
| **Composition** | B owns A — A cannot exist without B (strong) |
| **Aggregation** | B uses A — A lives independently (weak) |
| **Normalization** | Write-heavy, reduces redundancy |
| **Denormalization** | Read-heavy, allows duplication for speed |
| **Transaction** | BEGIN → SQL → COMMIT or ROLLBACK |
| **Serializable** | Highest isolation — no dirty/phantom reads |
| **Index** | Speeds reads on WHERE/JOIN cols — slows writes |

---

*Good luck — be direct, think out loud, admit when unsure rather than guessing.*